• ready - Highest priority thread that is ready enters running state
• running - System assigns a processor to the thread and the thread begins to execute its run() method. When run() method completes or terminates, thread enters dead state.
• dead - Thread marked to be removed from the system. Entered when run() terminates or throws uncaught exception.
• blocked - To enter blocked state, thread must already have been in running state. Even if the CPU is available, a blocked thread cannot use the processor. Common reason for a thread being in a blocked state—waiting on a request for file I/O .
• sleeping - Entered when sleep() method has been called for a thread. While in the sleep state, a thread cannot use the processor. After its sleep time expires, the thread re-enters the ready state. It doesn't starting running again, it becomes eligible to run—much in the same way it did when you called the start() method.
• waiting - Entered when wait() method has been called in an object thread is accessing. One waiting thread becomes ready when object calls the method notify(). When the method notifyAll() has been called—all waiting threads become ready.

A thread executes until:
• it dies,
• it calls sleep(),
• it calls wait(),
• it calls yield(),
• its quantum of time is used up, or
• it is preempted by a higher priority thread.

When a thread has just been instantiated, your only option is to start() it. If you try to execute any method besides start(), you will cause an

IllegalThreadStateException

When the start() method is returns the thread is ready to be run when the scheduler decides it's the right time. Notice on the right there is the state called "not runnable." There are several reasons why a thread that has not finished its run() method could find itself "not runnable." :
• If a thread has been put to sleep, then the sleep milliseconds must elapse.
• If a thread is waiting for something, then another object must notify the waiting thread of a change in condition. The other object would handle the notification by calling the methods notify() or notifyAll().
• If a thread is blocked on I/O, then the I/O must complete.